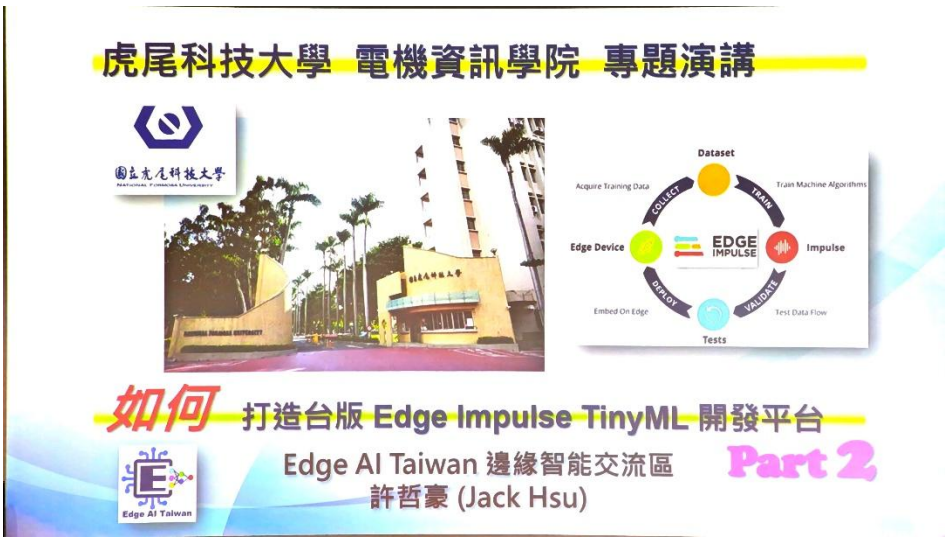
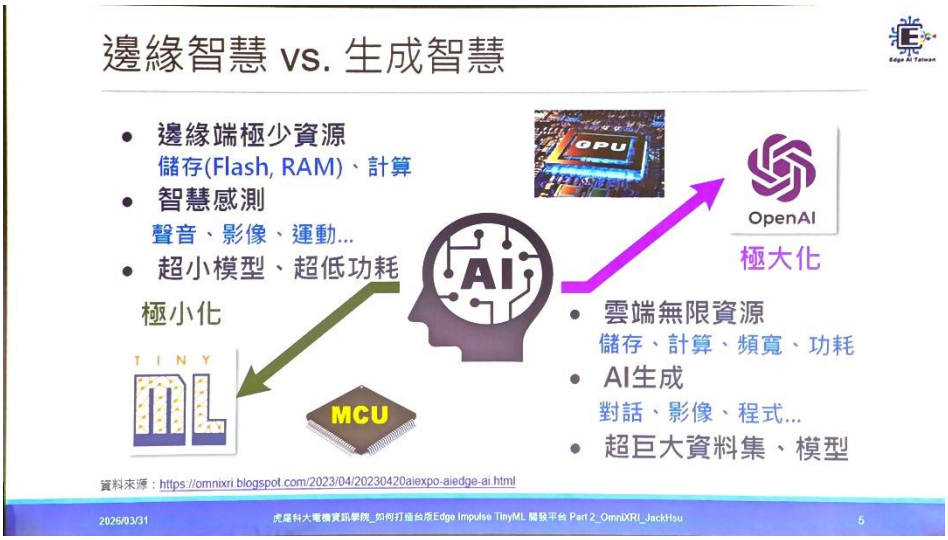
日期:2026/03/31

講師:許哲豪

題目:如何打造台版 Edge Impulse TinyML 開發平台





邊緣端資源很少，儲存空間、RAM、計算能力都有限

主要做的是感測型任務像是聲音、影像、運動

所以模型必須小型且低功耗

而**生成智慧**依賴雲端的大量資源

能對話、影像生成、程式生成

需要超大模型超大資料集，能源消耗高可能一個簡單任務就超過我們手機電量

AI 技術演進方向

感知 AI(影像辨識、語音辨識)、生成 AI(聊天、畫圖、寫程式)、代理 AI(執行任務)、物理 AI(機器人、自駕、感測控制)

三種尺度：

預訓練尺度、後訓練尺度、測試時尺度（長思考）

技術演進

CNN 影像

- 是早期電腦視覺代表模型

RNN

- 主要處理時序資料，例如聲音、文字序列

LLM

- 大語言模型
- 以文字理解與生成為主

VLM

- 視覺語言模型
- 結合圖片與文字

VLA

- 視覺語言動作模型再加上控制／動作

Edge AI 分成幾個層次

Edge Server

NB AI（AI PC）

Mobile AI（SoC）

SBC AI（MPU / NPU / GPU）

Wearable AI（MCU / TinyML）

Edge AI 並沒有單一定義，會因為哪一種硬體平台而有不同答案。

不同公司、不同產品線、不同應用場景，對 Edge AI 的理解不同。

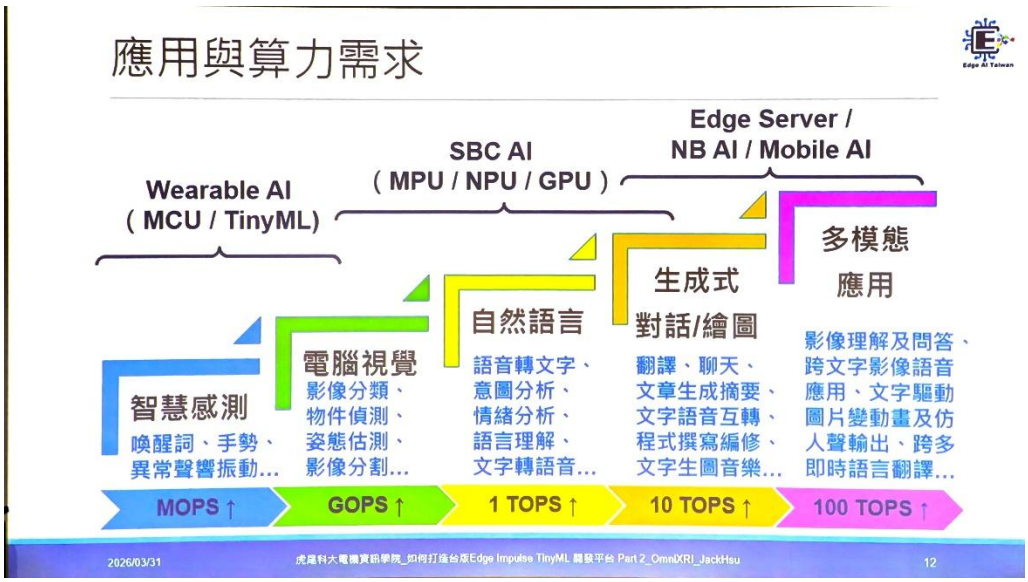
Edge AI 如何定義？

廣義定義

不上網就能完成所有 AI 推論。

狹義定義

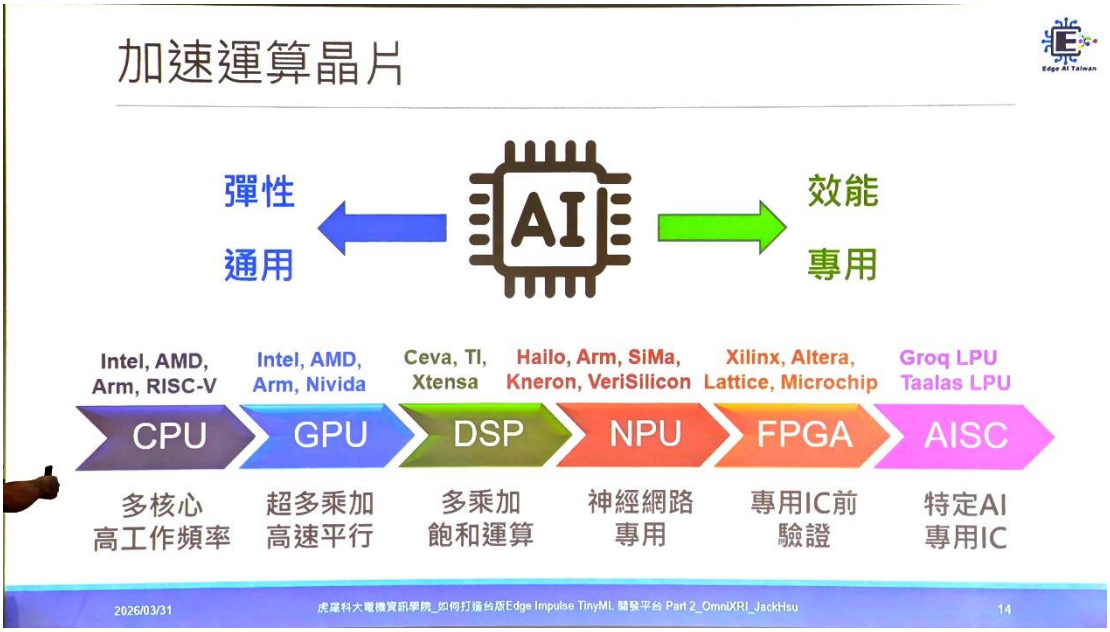
使用電池供電完成所有 AI 推論的獨立裝置。



應用與算力需求

不同 AI 應用，對算力需求差很多。

Edge AI 開發流程

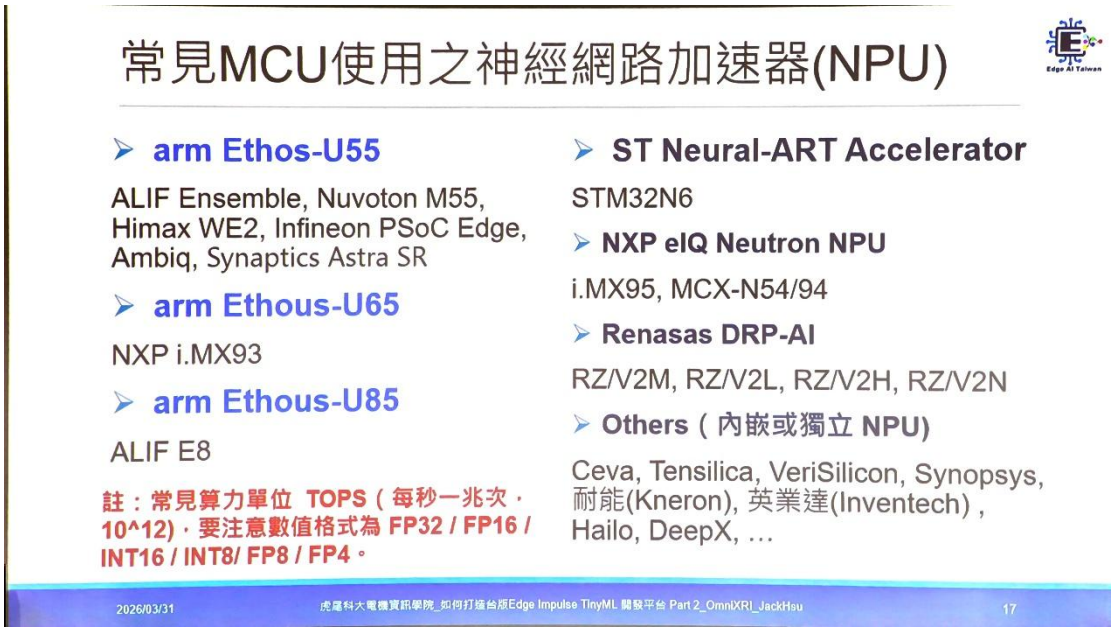


加速運算晶片

左邊偏彈性、通用

右邊偏效能、專用

從 DSP、NPU、FPGA 到專用 AI 晶片，都可能是不同應用的最佳選擇



常見 MCU 使用之神經網路加速器 NPU

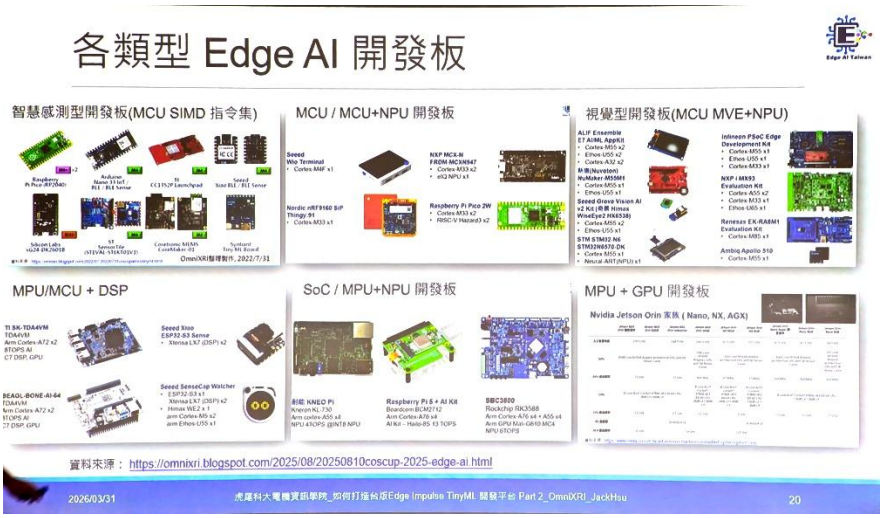
硬體加速方式與限制

算力牆:處理器本身不夠快。

記憶體牆:資料搬移太慢，記憶體頻寬不夠。

能耗牆:功耗太高，散熱或電池撐不住。

通訊牆:資料傳輸速度跟不上，系統整體效能被卡住。



開發板依照算力與應用型態分成幾個分別

智慧感測型開發板（MCU SIMD 指令集）、MCU / MCU+NPU 開發板、

視覺型開發板（MCU MVE+NPU）、MPU/MCU + DSP、SoC / MPU+NPU 開發板、MPU + GPU 開發板

主流 AI 開發工具

伺服器、桌機、筆電

TensorFlow

PyTorch

手機、平板、單板微電腦

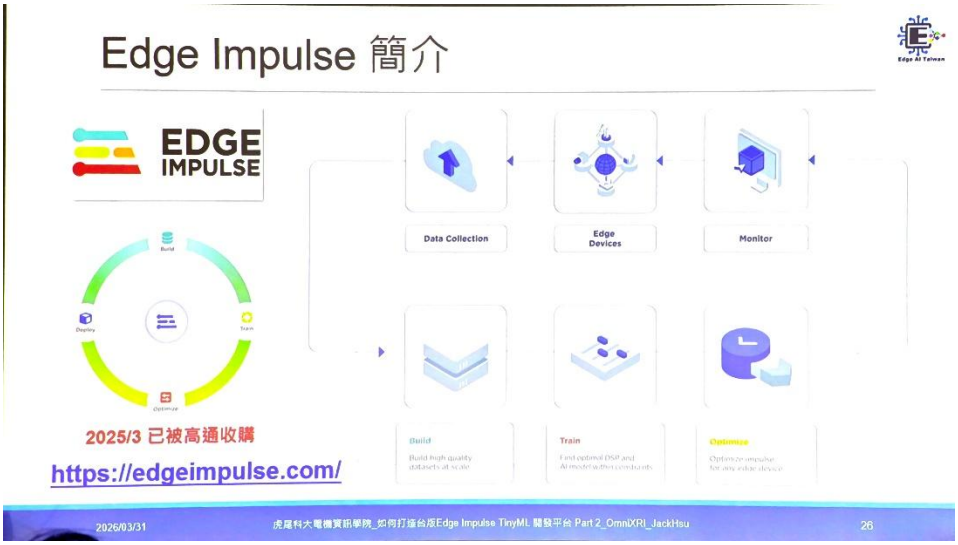
TensorFlow Lite（現在 LiteRT）

PyTorch for mobile

微控制器、單晶片

TensorFlow Lite for Microcontrollers

ExecuTorch



Edge Impulse 簡介

1. 收資料
2. 建資料集
3. 訓練模型
4. 模型最佳化
5. 佈署到裝置端

做成一個平台化工具降低 Edge AI 開發門檻

Edge Impulse 平台除了開發工具以外，還提供很多技術支援資源：

說明文件

共享專案

技術論壇

技術文章

Arduino、Qualcomm、Edge Impulse 正在整合

Edge AI 會越來越普遍

台灣需要自己的 Edge Impulse 類平台

提升嵌入式人工智慧工程韌性

加速 Edge AI 與 TinyML 開發效率

提供國內產官學研整合性實驗平台

共享資源、快速提升工程水準

高度整合國內晶片與軟體業者

Seeed Studio 的 SenseCraft AI

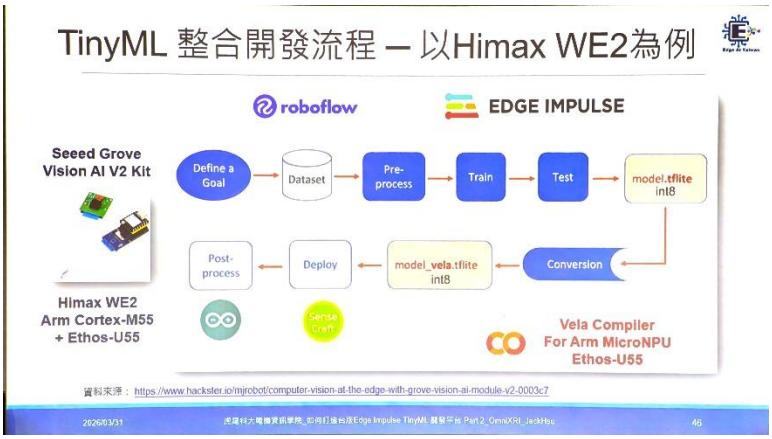
支援的硬體：

Grove Vision AI Kit V2、SenseCAP A1102、SenseCAP Watcher（Himax WE2）、XIAO ESP32S3、Nvidia Jetson、reCamera

主要以視覺為主

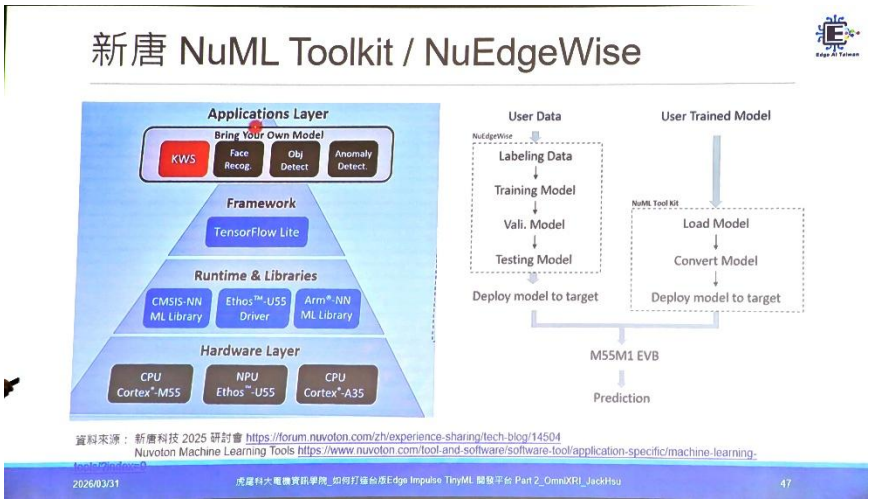
可以做影像分類、物件偵測

也提供資料集建置、模型訓練、快速部署等服務



TinyML 整合開發流程 實際硬體 **Himax WE2 + Ethos-U55**

1. Define a Goal
2. Dataset
3. Pre-process
4. Train
5. Test
6. 產出 model.tflite int8
7. Conversion
8. 轉成 model_vela.tflite int8
9. Deploy
10. Post-process



新唐 NuML Toolkit / NuEdgeWise　　新唐自己的工具鏈

左邊是分層架構：

- Applications Layer
- Framework：TensorFlow Lite
- Runtime & Libraries：
  - CMSIS-NN
  - Ethos-U55 Driver
  - Arm-NN ML Library
- Hardware Layer：
  - Cortex-M55
  - Ethos-U55
  - Cortex-A35

右邊則是流程：

1. 使用者資料
2. Labeling Data
3. Training Model
4. Validation
5. Testing
6. 或直接把使用者訓練好的模型 Load / Convert
7. 最後 Deploy 到 M55M1 EVB 做 Prediction

自建平台技術痛點

- **模型轉換（硬體加速）**

算子相容性、NPU 轉譯工具

不是所有模型都能直接丟到 NPU 上跑，常常會卡

- **模型壓縮（優化）**

量化、剪枝、蒸餾、權重共享

要把模型放到小裝置上，通常要做壓縮與最佳化。

- **自動產生模型（AutoML）**

Neural Network Search（NAS）

自動產生最優模型結構

平台想更好用，不能只讓使用者手刻模型，還要有適合的模型結構提供。

- **訓練加速**

配合高速 GPU 或使用遷移學習

即使最後模型要跑在小裝置，前端訓練階段還是很需要大算力支援。